CHRONO SUPPORT FOR HUMAN-IN-THE-LOOP SIMULATION

Speaker: Kyle Sha
(Simulation-Based Engineering Lab)

Overview of the human-in-the-loop tutorial

- Human-in-the-loop = the simulation runs at wall-clock speed and a person closes the control loop
- In this tutorial, you will learn how to
  - PART 1: Keep a Chrono::Vehicle simulation real-time and measure how well you did
  - PART 2: Put a human on the keyboard with ChInteractiveDriver
  - PART 3: Read a gamepad or wheel natively - no extra library needed
  - PART 4: Bring a device Chrono doesn't know about, and close the loop back to it
  - PART 5 (bonus): Customize the built-in overlay and choose your car -- cosmetic, not the core lesson
- Everything runs in PyChrono; no special hardware is required (a gamepad/wheel is optional)
- The whole human-in-the-loop contract is: spin once per step, three floats in, state out

Before we start ……

- Create a conda environment with PyChrono and pygame
  - `conda create -n chrono_hil -c projectchrono -c conda-forge pychrono pygame python=3.13`
  - `conda activate chrono_hil`
- Do NOT `pip install pychrono` - the PyPI package with that name is an unrelated library
- Get the tutorial files: tutorial_HIL_driver.py, operator_console.py
- Check that `python tutorial_HIL_driver.py` opens an Irrlicht window with an HMMWV
- All switches are in the CONFIGURATION section at the bottom of tutorial_HIL_driver.py

Part 1: Keep the simulation real-time

What "real-time" means in Chrono

- Chrono integrates as fast as it can; it has no notion of wall-clock time by itself
- Real Time Factor: RTF = compute time for a step / step size
  - RTF < 1: faster than real time - we must WAIT after each step
  - RTF > 1: slower than real time - nothing can be done except a cheaper model or a bigger step
- Soft real-time: spin after each step until wall time catches up with simulation time
  - No OS scheduling guarantees; good enough for a driving simulator, not for a control ECU
- Three equivalent mechanisms in PyChrono (next slides): `ChRealtimeStepTimer`, `ChVehicle.EnableRealtime`, or your own

Configuration: scripted driver, as fast as possible (Lines 435 to 453)

```
# Where do the driver inputs come from?
#   "data"      PART 1 - scripted ChDataDriver, no human
#   "keyboard"  PART 2 - ChInteractiveDriver, arrow keys in the Irrlicht window
#   "gamepad"   PART 3 - a joystick/wheel, read natively (JOYSTICK_CONFIG below)
#   "udp"       PART 4 - operator_console.py sends packets over the network
INPUT_SOURCE = "data"

# How is real time enforced?  "none" | "per_step" | "vehicle" | "cumulative"
REALTIME = "none"

# PART 4: send telemetry back to the operator (udp source only)
SEND_FEEDBACK = False

# Simulation step sizes.  Make step_size smaller (e.g. 5e-4) to see RTF > 1.
step_size = 3e-3
tire_step_size = 1e-3

# Tire model (RIGID, FIALA, TMEASY, PAC89, PAC02)
tire_model = veh.TireModelType_TMEASY
```

Start with INPUT_SOURCE = "data" and REALTIME = "none"
A ChDataDriver replays a table of (time, steering, throttle, braking) - no human yet

Step 1: A scripted ChDataDriver (Lines 284 to 294)

```
if INPUT_SOURCE == "data":
    ### PART 1: scripted inputs - no human in the loop ###
    # (time, steering, throttle, braking)
    data = veh.vector_Entry([
        veh.DataDriverEntry(0.0, 0.0, 0.0, 0.0),
        veh.DataDriverEntry(0.5, 0.0, 0.8, 0.0),
        veh.DataDriverEntry(4.0, 0.4, 0.8, 0.0),
        veh.DataDriverEntry(8.0, -0.4, 0.8, 0.0),
        veh.DataDriverEntry(12.0, 0.0, 0.0, 0.8),
    ])
    driver = veh.ChDataDriver(vehicle, data)
```

ChDataDriver interpolates the entries in time

Every driver in Chrono::Vehicle produces the same thing: DriverInputs = (steering, throttle, braking, clutch)

Step 2: The simulation loop (Lines 376 to 404)

```
while vis.Run():
    sim_time = system.GetChTime()

    # Render scene
    if step_number % render_steps == 0:
        vis.BeginScene()
        vis.Render()
        vis.EndScene()

    # PART 4: sample the device ONCE per step and hold it for the step
    if device is not None:
        smoother.set_target(*device.poll())
        smoother.advance(step_size)
        smoother.apply_to(driver)

    # Get driver inputs (three floats) - this is the whole human-in-the-loop contract
    driver_inputs = driver.GetInputs()

    # Update modules (process inputs from other modules)
    driver.Synchronize(sim_time)
    terrain.Synchronize(sim_time)
    vehicle_model.Synchronize(sim_time, driver_inputs, terrain)
    vis.Synchronize(sim_time, driver_inputs)

    # Advance simulation for one timestep for all modules
    driver.Advance(step_size)
    terrain.Advance(step_size)
    vehicle_model.Advance(step_size)  # spins here if REALTIME == "vehicle"
    vis.Advance(step_size)
```

Synchronize: every module reads what it needs from the others at time t

Advance: every module integrates from t to t + step

The driver is queried BEFORE Synchronize - inputs are sampled once and held for the step

Step 3: Measure it - sim time vs wall time (Lines 406 to 412)

```
# Console report: how far is the sim from wall time?
if step_number % report_steps == 0:
    wall = time.perf_counter() - wall_start
    print(f"{sim_time:7.2f} {wall:7.2f} {wall - sim_time:+7.3f} {vehicle.GetRTF():6.2f}"
          f" {vehicle.GetSpeed():7.2f}  {driver_inputs.m_steering:6.2f}"
          f" {driver_inputs.m_throttle:5.2f} {driver_inputs.m_braking:5.2f}")
```

drift = wall time - simulation time: negative means the sim is ahead of the clock

vehicle.GetRTF() is the true RTF of the last step, even when real time is enforced

Show Time

- REALTIME = "none": watch the drift column run away
- Change step_size to 5e-4 - RTF goes above 1 and the sim can never be real-time

Step 4: Enforce real time, three ways (Lines 357 to 362)

```
# Real-time setup (PART 1)
# ------------------------------------------------------------------
if REALTIME == "vehicle":
    vehicle.EnableRealtime(True)
rt_timer = chrono.ChRealtimeStepTimer()  # used when REALTIME == "per_step"
cum_timer = CumulativeRealtimeTimer()    # used when REALTIME == "cumulative"
```

"vehicle": ChVehicle::EnableRealtime(True) spins inside vehicle.Advance() using a ChRealtimeStepTimer
"per_step": the same timer, called explicitly at the end of the loop (next slide)
"cumulative": our own timer that compares total simulated time to total wall time

Step 4: Spin at the end of the loop (Lines 421 to 425)

```
# PART 1: spin in place for real time to catch up
if REALTIME == "per_step":
    rt_timer.Spin(step_size)
elif REALTIME == "cumulative":
    cum_timer.spin(system.GetChTime())
```

ChRealtimeStepTimer.Spin(step): wait until at least `step` seconds passed since the last Spin, then reset

Per-step timers never recover lost time: a slow frame, a Python hiccup, a render - each one pushes drift up a little

Step 4: A drift-free timer in 10 lines (Lines 56 to 74)

```
class CumulativeRealtimeTimer:
    """Soft real-time that does not accumulate drift.

    ChRealtimeStepTimer measures each step independently: any time lost on a
    slow step (or spent in Python between steps) is never recovered.  This timer
    instead compares the *total* simulated time with the *total* wall time, so a
    slow step is followed by a few fast steps that catch back up.
    """

    def __init__(self):
        self.reset()

    def reset(self):
        self.t_start = time.perf_counter()

    def spin(self, sim_time):
        while time.perf_counter() - self.t_start < sim_time:
            pass  # spin in place until real time catches up
```

Compare TOTAL simulated time with TOTAL wall time - lost time is caught up on the next fast steps
This is how driving simulators (e.g. Chrono::HIL) keep their clock; the same idea also works across processes

Before running the code, please

- Set REALTIME = "per_step" (Line 389), run for a while and note the drift
- Set REALTIME = "cumulative" and compare
- Keep REALTIME = "vehicle" for the rest of the tutorial - it is the simplest

Show Time

- Measured on a laptop, HMMWV + TMeasy tires, step 3 ms (RTF ~ 0.14):
  - "none": drift -69 s after 17 s of wall time
  - "per_step": drift +0.012 s after 17 s and growing
  - "vehicle": drift +0.021 s after 17 s and growing
  - "cumulative": drift +0.000 s
- Drift matters more the closer you are to RTF = 1

Part 2: A human on the keyboard

Step 1: ChInteractiveDriver (Lines 296 to 307)

```
elif INPUT_SOURCE == "keyboard":
    ### PART 2: keyboard through the Irrlicht window ###
    # Arrow keys steer/accelerate/brake, 'C' centers steering, 'R' releases
    # the pedals, 'L' locks the current inputs.
    driver = veh.ChInteractiveDriver(vehicle)
    steering_time = 1.0  # time to go from 0 to +1 (or from 0 to -1)
    throttle_time = 1.0  # time to go from 0 to +1
    braking_time = 0.3   # time to go from 0 to +1
    driver.SetSteeringDelta(render_step_size / steering_time)
    driver.SetThrottleDelta(render_step_size / throttle_time)
    driver.SetBrakingDelta(render_step_size / braking_time)
    driver.SetGains(4.0, 4.0, 4.0)  # first-order lag from key target to applied input
```

Arrow keys steer/accelerate/brake, C centers steering, R releases the pedals, L locks the inputs
Each keypress moves a TARGET by "delta"; the applied input follows the target with a first-order lag (SetGains)
delta = render_step / time-to-full-scale: full steering lock takes 1 s, full braking 0.3 s

Step 2: Route the window events to the driver (Lines 339 to 342)

```
vis.AddSkyBox()
vis.AttachVehicle(vehicle)
if INPUT_SOURCE in ("keyboard", "gamepad"):
    vis.AttachDriver(driver)  # route Irrlicht key/joystick events to the driver
```

The Irrlicht visual system owns the keyboard; AttachDriver forwards key events to the driver
Nothing else in the loop changes - the driver still just returns three floats

Before running the code, please

- Set INPUT_SOURCE = "keyboard" (Line 386)
- Set REALTIME = "vehicle" (Line 389) - without real time the car is undrivable (why?)
- Try SetGains(50, 50, 50) on Line 275 and feel what a step input does to the suspension

Show Time

- Volunteer: drive the HMMWV. Everyone else: watch the Irrlicht HUD (speed, inputs, RTF)

Part 3: A gamepad or wheel, read natively

No pygame, no custom class - Chrono reads the joystick

- ChInteractiveDriver already knows how to read a joystick: `SetInputMode(JOYSTICK)`
- A JSON config file maps device axes/buttons to steering/throttle/braking/clutch/shifting - four presets ship with Chrono in `data/vehicle/joystick/`
  - `controller_XboxOneForWindows.json`
  - `controller_LogitechRumblePad2.json`
  - `controller_WheelPedalsAndShifters.json`
  - `controller_Default.json` - or write your own
- `SetJoystickDebug(True)` prints live axis/button numbers twice a second - the same job probe_gamepad.py used to do, built in

The gamepad/wheel driver (Lines 309 to 316)

```
elif INPUT_SOURCE == "gamepad":
    ### PART 3: a gamepad or wheel - same driver class as keyboard ###
    # SetInputMode(JOYSTICK) switches ChInteractiveDriver from reading the
    # Irrlicht window's keyboard to reading the joystick vis configures
    # below.  Smoothing (SetGains) applies the same way either mode.
    driver = veh.ChInteractiveDriver(vehicle)
    driver.SetGains(4.0, 4.0, 4.0)
    driver.SetInputMode(driver.InputMode_JOYSTICK)
```

Same ChInteractiveDriver class as keyboard - just a different input mode

Wire up the joystick (Lines 341 to 345)

```
if INPUT_SOURCE in ("keyboard", "gamepad"):
    vis.AttachDriver(driver)  # route Irrlicht key/joystick events to the driver
if INPUT_SOURCE == "gamepad":
    vis.SetJoystickConfigFile(JOYSTICK_CONFIG)
    vis.SetJoystickDebug(JOYSTICK_DEBUG)  # prints live axis/button numbers
```

JOYSTICK_CONFIG and JOYSTICK_DEBUG live in CONFIGURATION, next to VEHICLE

Before running the code, please

- Set INPUT_SOURCE = "gamepad" (Line 440), keep REALTIME = "vehicle"
- Plug in a gamepad or wheel; set JOYSTICK_CONFIG to the preset that matches it
- No matching preset? Set `JOYSTICK_DEBUG = True` first and move one control at a time to read off the axis/button numbers

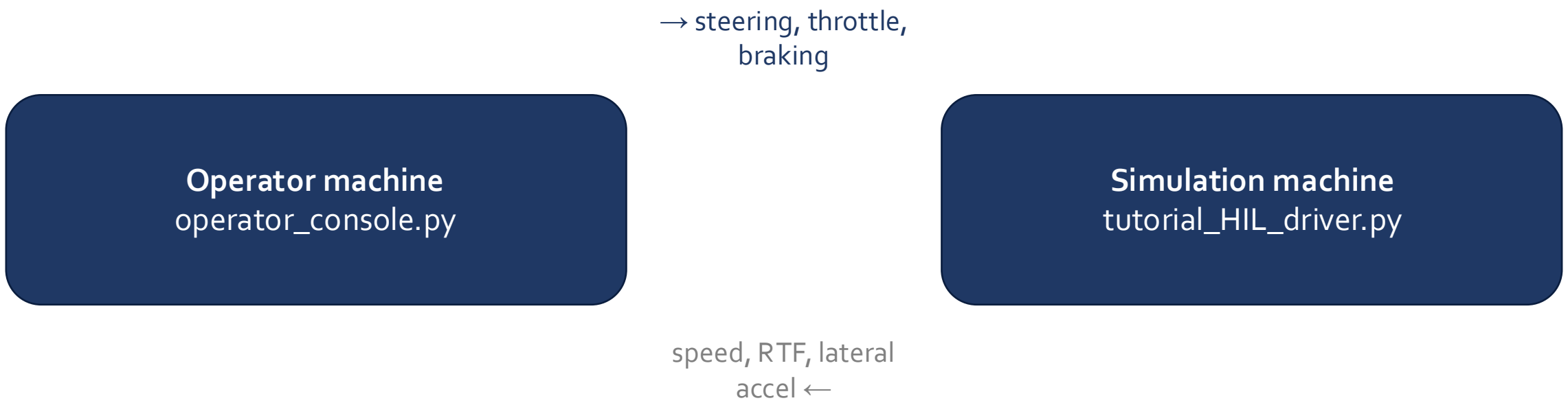
Show Time

- Volunteer: drive the HMMWV with a gamepad or wheel - no second terminal, no pygame window
- Compare the feel against Part 2's keyboard: same SetGains() smoothing, continuous analog input instead of discrete keypresses

Part 4: A device Chrono doesn't know about, and closing the loop

a UDP console, a gamepad, a steering wheel, a phone, another process…

Part 4's architecture: two processes, one UDP contract



Both directions are plain UDP datagrams - two processes, possibly two machines, no shared memory. The grey arrow only fires when SEND_FEEDBACK = True.

The human-in-the-loop contract

- A driver is just three floats per step: steering [-1, 1], throttle [0, 1], braking [0, 1]
- `veh.ChDriver` is the plain container: `SetSteering()`, `SetThrottle()`, `SetBraking()`
- So the recipe for ANY device is:
  - 1. poll the device (never block the physics loop)
  - 2. rate-limit / smooth (a human arm is not a step function)
  - 3. write into the ChDriver once per step - zero-order hold until the next step
- Today: a UDP operator console everybody can run (two terminals)

Step 1: Smooth - never teleport raw inputs into the model (Lines 100 to 133)

```
class SmoothedInputs:
    """First-order lag from a *target* input to the *applied* input.

    Raw device values should not be teleported into the model: a keypress is a
    step function, and a step in steering excites the suspension and tires in a
    way no human arm ever would.  This mirrors the internal dynamics of
    ChInteractiveDriver (see SetGains) so the behavior is the same whether the
    inputs come from the Irrlicht keyboard handler or from our own device.
    """

    def __init__(self, gain=4.0):
        self.gain = gain
        self.steering = 0.0
        self.throttle = 0.0
        self.braking = 0.0
        self.target = (0.0, 0.0, 0.0)

    def set_target(self, steering, throttle, braking):
        self.target = (
            max(-1.0, min(1.0, steering)),
            max(0.0, min(1.0, throttle)),
            max(0.0, min(1.0, braking)),
        )

    def advance(self, step):
        s, t, b = self.target
        self.steering += min(1.0, self.gain * step) * (s - self.steering)
        self.throttle += min(1.0, self.gain * step) * (t - self.throttle)
        self.braking += min(1.0, self.gain * step) * (b - self.braking)

    def apply_to(self, driver):
        driver.SetSteering(self.steering)
        driver.SetThrottle(self.throttle)
        driver.SetBraking(self.braking)
```

Same first-order lag as ChInteractiveDriver::SetGains, in Python, so all devices behave alike

Step 2: A UDP input source (Lines 136 to 169)

```
class UdpInput:
    """Receive 'steering,throttle,braking' text datagrams from an operator.

    Run operator_console.py in another terminal (or on another machine) and
    drive with the arrow keys there.  The socket is non-blocking: the physics
    loop never waits for the operator.  If no packet arrived since the last
    step, the previous target is held (zero-order hold).
    """

    def __init__(self, port=9870):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(("0.0.0.0", port))
        self.sock.setblocking(False)
        self.operator_addr = None
        self.last = (0.0, 0.0, 0.0)
        self.packets = 0
        print(f"[udp] listening on port {port} - start operator_console.py")

    def poll(self):
        # Drain everything that is queued and keep only the newest packet
        while True:
            try:
                data, addr = self.sock.recvfrom(256)
            except BlockingIOError:
                break
            try:
                s, t, b = (float(x) for x in data.decode().split(","))
                self.last = (s, t, b)
                self.operator_addr = addr
                self.packets += 1
            except ValueError:
                pass
        return self.last
```

Non-blocking socket, drained every step; the newest packet wins, an empty queue holds the last value
The operator can be anywhere on the network - and the console has no Chrono dependency at all

Step 3: The operator side - operator_console.py

```
# --- keyboard -> target inputs ----------------------------------------
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]:
    steering = min(1.0, steering + STEER_RATE * dt)     # +1 = left in Chrono
elif keys[pygame.K_RIGHT]:
    steering = max(-1.0, steering - STEER_RATE * dt)
else:  # self-center
    steering -= max(-STEER_RETURN * dt, min(STEER_RETURN * dt, steering))
if keys[pygame.K_UP]:
    throttle = min(1.0, throttle + PEDAL_RATE * dt)
    braking = 0.0
elif keys[pygame.K_DOWN]:
    braking = min(1.0, braking + PEDAL_RATE * dt)
    throttle = 0.0
else:
    throttle = max(0.0, throttle - PEDAL_RATE * dt)
    braking = max(0.0, braking - PEDAL_RATE * dt)
if keys[pygame.K_SPACE]:
    steering, throttle, braking = 0.0, 0.0, 0.0

# --- send to the simulation -------------------------------------------
sock.sendto(f"{steering:.3f},{throttle:.3f},{braking:.3f}".encode(), (SIM_HOST, SIM_PORT))
```

A pygame window: hold the arrow keys, send `steering,throttle,braking` at 50 Hz
Swap this for a web page, a ROS node, or a driving-simulator cockpit - the sim does not care

Step 4: Plug the device into a plain ChDriver (Lines 318 to 327)

```
else:
    ### PART 4: a device Chrono doesn't know about writes into a plain ChDriver ###
    driver = veh.ChDriver(vehicle)
    if INPUT_SOURCE == "udp":
        device = UdpInput(port=UDP_PORT)
    else:
        raise ValueError(f"unknown INPUT_SOURCE {INPUT_SOURCE!r}")
    smoother = SmoothedInputs(gain=4.0)

driver.Initialize()
```

No AttachDriver here - the Irrlicht window keeps the keyboard, our device owns the inputs

Step 5: Sample once per step, then hold (Lines 384 to 393)

```
# PART 4: sample the device ONCE per step and hold it for the step
if device is not None:
    smoother.set_target(*device.poll())
    smoother.advance(step_size)
    smoother.apply_to(driver)

# Get driver inputs (three floats) - this is the whole human-in-the-loop contract
driver_inputs = driver.GetInputs()
```

poll -> smooth -> write, then GetInputs() as before; the physics loop never waits on the device

Before running the code, please

- Set INPUT_SOURCE = "udp" (Line 386), keep REALTIME = "vehicle"
- Terminal 1: `python tutorial_HIL_driver.py`
- Terminal 2: `python operator_console.py` (or `python operator_console.py <ip of terminal 1>` from another laptop)
- Click the operator console window and drive with the arrow keys

Show Time

- Pair up: one laptop runs the sim, the other runs the console and drives it over Wi-Fi
- Stop the console mid-drive: the last input is held - what should a real simulator do here?

Step 1: Send state back at render rate (Lines 413 to 417)

```
# PART 4: feedback to the operator (speed, RTF, lateral acceleration, applied inputs)
if SEND_FEEDBACK and device is not None and step_number % render_steps == 0:
    acc = vehicle.GetPointAcceleration(chrono.ChVector3d(0, 0, 0))
    device.send_feedback(f"{sim_time:.3f},{vehicle.GetSpeed():.3f},{vehicle.GetRTF():.3f},{acc.y:.3f},"
                         f"{driver_inputs.m_steering:.3f},{driver_inputs.m_throttle:.3f},{driver_inputs.m_braking:.3f}")
```

Speed, RTF and lateral acceleration go back to whoever sent us inputs (UdpInput remembers the address)

Send at render rate (50 Hz), not at every physics step - the human cannot use 333 Hz

Step 2: Show it on the console - operator_console.py

```
# --- PART 4: receive telemetry ----------------------------------------
while True:
    try:
        data, _ = sock.recvfrom(256)
        telemetry = tuple(float(x) for x in data.decode().split(","))
        # (sim time, speed, RTF, lateral acc, applied steering, throttle, braking)
    except (BlockingIOError, ValueError):
```

Before running the code, please

- Set SEND_FEEDBACK = True (Line 392)
- Restart both scripts; the console now shows speed, sim time, RTF and lateral acceleration
- With a wheel: this is where force feedback torque would be computed from the lateral acceleration

Show Time

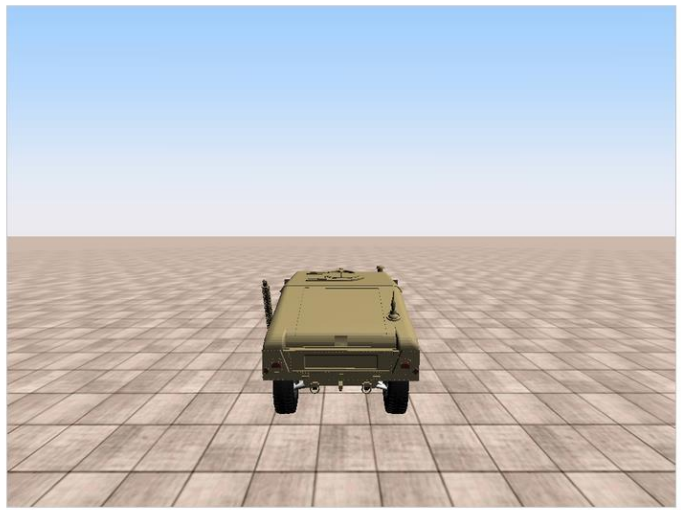
Part 5: Choose your car, customize the overlay

Bonus material: the car picker and overlay flags are nice-to-haves. Parts 1 and 4 are the patterns worth taking with you.

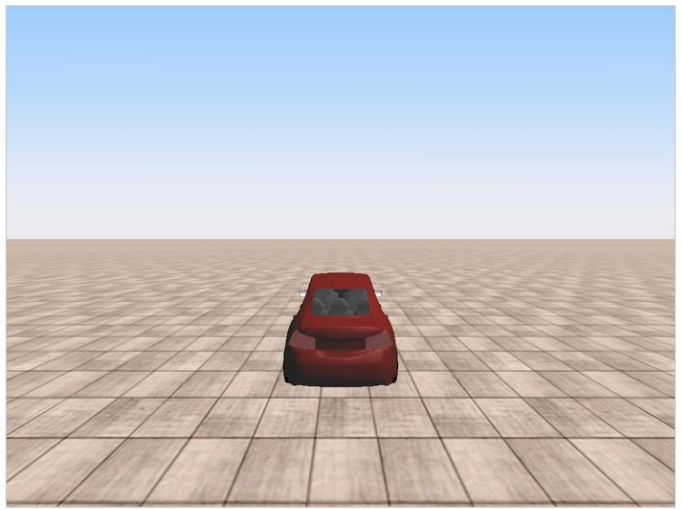
One interface, many vehicles - and a HUD you don't have to build

- Every full-vehicle model in Chrono::Vehicle (HMMWV_Full, Sedan, UAZBUS, Gator, …) shares the same interface
- `GetVehicle()` / `GetSystem()` / `Synchronize()` / `Advance()` - the same four calls no matter which model you picked
- VEHICLE picks the model; build_vehicle() is the only place that knows the differences (init height, extra setup calls)
- The Irrlicht window already ships with an on-screen HUD, plus Chrono's own tabbed info panel and a profiler
- None of it is hand-drawn - add or remove pieces with flags on vis, no custom overlay needed

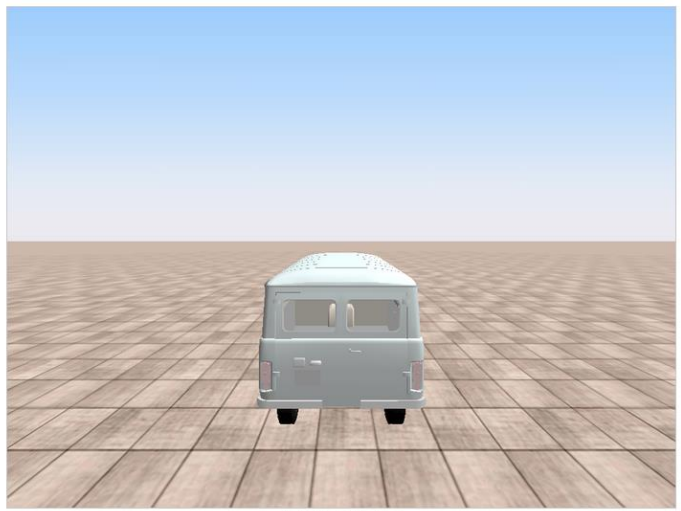
What you'll actually see



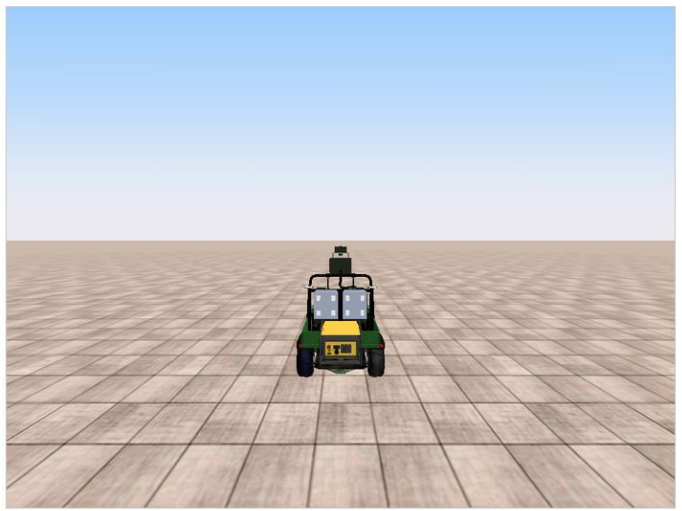
HMMWV



Sedan



UAZBUS



Gator

Choose your car (Lines 256 to 264)

```
def main():
    # -----------------------------------------------------------------
    # Create the vehicle (PART 5) and initialize
    # -----------------------------------------------------------------
    vehicle_model, chase_dist = build_vehicle(VEHICLE)

    vehicle = vehicle_model.GetVehicle()
    system = vehicle_model.GetSystem()
    system.SetCollisionSystemType(chrono.ChCollisionSystem.Type_BULLET)
```

VEHICLE lives in CONFIGURATION - "hmmwv" | "sedan" | "uazbus" | "gator"; everything below this line is unchanged

Add or remove overlay pieces (Lines 347 to 354)

```
# PART 5: the built-in overlay - nothing here is hand-drawn, every piece
# is a flag or a method on `vis` itself
vis.EnableStats(SHOW_VEHICLE_HUD)        # speed/steering/throttle/brake panel
vis.SetHUDLocation(*HUD_CORNER)          # where that panel sits
vis.ShowInfoPanel(SHOW_SIM_INFO_PANEL)   # Chrono's own tabbed info panel
                                          # (bodies, contacts, timers); also
                                          # toggles live with the 'i' key
vis.ShowProfiler(SHOW_PROFILER)          # per-module timing bars
```

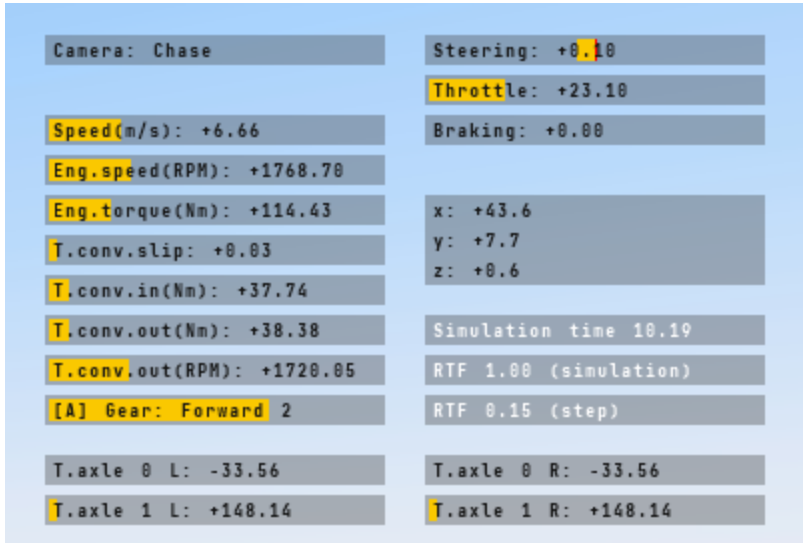
SHOW_VEHICLE_HUD, HUD_CORNER, SHOW_SIM_INFO_PANEL, SHOW_PROFILER all live in CONFIGURATION, right next to VEHICLE

Before running the code, please

- Set VEHICLE = "uazbus" (or "sedan", "gator") and re-run - same driver, same terrain, very different vehicle
- Set SHOW_SIM_INFO_PANEL = True, then press the `i` key while the sim is running - same panel, two ways to reach it
- Try SHOW_VEHICLE_HUD = False - the 3D scene doesn't care, only the panel disappears

Show Time

- Everyone picks a different VEHICLE and compares how the same scripted course drives an HMMWV vs. a bus vs. a Gator
- Toggle SHOW_VEHICLE_HUD / SHOW_SIM_INFO_PANEL live and watch the overlay change - nothing was re-rendered by hand



Where this goes next

- Chrono::HIL (github.com/zzhou292/chrono-HIL): the same three-float interface, scaled up
  - SDL steering wheels with force feedback, cumulative real-time timer, multi-vehicle traffic
  - ROS 2 and Unity bridges, teleoperation over the network, deformable (SCM) terrain
- Chrono::Sensor: cameras/lidar rendered from the vehicle for the operator's screen
- Chrono::Synchrono: several humans in the same world, each on their own machine
- Takeaways
  - Real time in Chrono is soft: spin once per step, and measure the drift
  - The human interface is three floats per step, sampled once and smoothed
  - Never let the input device block the physics loop

Any questions?

Thank you.

kasha2@wisc.edu
Simulation-Based Engineering Lab
University of Wisconsin - Madison

Lab website: http://sbel.wisc.edu
Chrono website: http://www.projectchrono.org
Source code: https://github.com/projectchrono